```
"""Top module importing everything and handling whole cities."""

from __future__ import division

from sets import Set                    # python 2.3 library

# Auto-reload all other modules with this.
def ir(module_name):
    """Import a module, reloading if already loaded."""
    import sys
    if module_name in sys.modules:
        reload(sys.modules[module_name])
    globals()[module_name] = __import__(module_name, globals(), locals(), [])

ir('utils')
ir('simtime')
ir('connect')
ir('basic')
ir('multi')
ir('textgrid')


class City(object):

    """A city - one set of connected simulated objects with common simtime.

    You should create a city object and use its Car, Source, etc.
    members to populate it...

    Change c.make_queue to control the queues used by MultiQueue objects
    (default - self.StatQueue).

    It's convenient to generate cities from a plan and in such case extra
    attributes of the city (.roads, .servers, .drains, etc.) will be set to
    details about it.

    """

    def __init__(self):
        """Borrows all classes from basic & mutli and wraps Timed ones."""
        import weakref
        self.weakself = weakref.proxy(self)
        self.simtime = simtime.SimTime(multi.phases)
        self.register = self.simtime.register
        self.unregister = self.simtime.unregister
        self.advance = self.simtime.advance
        def wrapclass(cl):
            """If cl is Timed, create wrapper that registers objects."""
            if not issubclass(cl, simtime.Timed):
                return cl
            def wrapper(*args, **opts):
                """Wrapper around a Timed class."""
                obj = object.__new__(cl)
                self.simtime.register(obj)
                obj.city = self.weakself
                obj.__init__(*args, **opts)
                return obj
            return wrapper
        for module in [basic, multi]:
            for name in dir(module):
                thing = getattr(module, name)
                if isinstance(thing, type):
                    setattr(self, name, wrapclass(thing))
        self.make_queue = self.StatQueue


# Coordinate utilities
def sub2((x1, y1), (x2, y2)):
    """Subtract 2 tuples element-wise."""
    return x1 - x2, y1 - y2
```

```
def neg2((x, y)):
    """Negate a tuple element-wise."""
    return -x, -y

def sort2(a, b):
    """Return min(a, b), max(a, b)."""
    return min(a, b), max(a, b)

# Direction literals
R, U, L, D = DIRECTIONS = (1, 0), (0, 1), (-1, 0), (0, -1)


class GridPlan(object):

    """Plan of a city where junctions are located on a grid.

    Junctions are placed on integral coordinates starting from (0, 0).  They
    have up to 4 roads going left, right, up and down.  All roads are
    bi-directional (i.e. represented by two `Road`s).  For simplicity, all
    roads are of length 1 and connect adjacent points.  Longer roads are
    simulated by passing through "forwarder" points (created with two
    `Road(0)` elements).

    The plan can be implemented (i.e. city is created according to it).

    The `p.roads` attribute is a set of sorted pairs of end-point coordinates
    to the roads.  The `p.grid` attribute is a dictionary mapping from (x, y)
    junction coordinates to sets of ports of each junction.

    The labels of ports are the directions going out of the junction, into the
    roads (e.g. a right-going road begins with `(1,0)` and ends with
    `(-1, 0)`).

    """

    def __init__(self, xsize, ysize):
        self.xsize, self.ysize = xsize, ysize
        self.xrange, self.yrange = range(xsize), range(ysize)
        self.xpairs = zip(self.xrange[:-1], self.xrange[1:])
        self.ypairs = zip(self.yrange[:-1], self.yrange[1:])
        self.roads = Set()
        self.grid = {}

    def add_road(self, p1, p2):
        """Add a road between two points."""
        p1, p2 = sort2(p1, p2)
        if (p1, p2) not in self.roads:
            self.roads.add((p1, p2))
            self.grid[p1].add(sub2(p2, p1))
            self.grid[p2].add(sub2(p1, p2))

    def del_road(self, p1, p2):
        """Remove a road between two points."""
        p1, p2 = sort2(p1, p2)
        if (p1, p2) in self.roads:
            self.roads.discard((p1, p2))
            self.grid[p1].discard(sub2(p2, p1))
            self.grid[p2].discard(sub2(p1, p2))

    def full_plan(self):
        """Plans a full grid of given size."""
        for x in self.xrange:
            for y in self.yrange:
                self.grid[x, y] = Set()

        for x in self.xrange:
            for y1, y2 in self.ypairs:
                self.add_road((x, y1), (x, y2))
        for y in self.yrange:
```

```
            for x1, x2 in self.xpairs:
                self.add_road((x1, y), (x2, y))

    def sieve_roads(self, prob):
        """Removes each road with probability prob."""
        import random
        for road in list(self.roads):
            if random.random() < prob:
                self.del_road(*road)

    def remove_perimeter(self):
        """Remove roads on the very perimeter => guarantee sources/sinks."""
        for x in (0, self.xsize-1):
            for y1, y2 in self.ypairs:
                self.del_road((x, y1), (x, y2))
        for y in (0, self.ysize-1):
            for x1, x2 in self.xpairs:
                self.del_road((x1, y), (x2, y))

    def implement(self, city, source, drain, server, router):

        """Create in `city` roads, sources, servers, etc. according to plan.

        `server(ports)` / `router(ports)` are called to create junctions with
        3 or 4 connections (the list of ports is provided in case they need it
        but they need not do the `ports > s`...).  With 2 connections a pair
        of road[0] forwarders is created to join the roads; with 1 connection
        a `source()` / `drain()` pair is created and with zero connections
        nothing is created.  Note that you should use the city's wrapped
        classes for the callable params.

        Various attributes (`c.sources`, etc.) are set on the city.

        """

        city.plan = self
        city.grid = {}
        city.roads = {}
        city.sources = {}
        city.drains = {}
        city.servers = {}
        city.routers = {}

        # handle junctions depending on number of connections
        for (x, y), dirs in self.grid.items():
            dirs = list(dirs)
            n = len(dirs)
            if n == 1:
                # source / drain
                city.sources[x, y] = source()
                city.drains[x, y] = drain()
                city.grid[x, y] = ((city.sources[x, y] | dirs[0]) +
                                   (dirs[0] | city.drains[x, y]))
            elif n == 2:
                # forwarder node
                city.grid[x, y] = ((dirs[0] | city.Road(0) | dirs[1]) +
                                   (dirs[1] | city.Road(0) | dirs[0]))
            elif n >= 3:
                # junction
                city.servers[x, y] = dirs > server(dirs)
                city.routers[x, y] = router(dirs) > dirs
                city.grid[x, y] = city.servers[x, y] | city.routers[x, y]

        # create roads and connect everything
        for ((x1, y1), (x2, y2)) in self.roads:
            dx, dy = x2 - x1, y2 - y1
            road = (dx, dy) | city.Road(1) | (-dx, -dy)
            city.grid[x1, y1] | road | city.grid[x2, y2]
            city.roads[(x1, y1), (x2, y2)] = road
            road = (-dx, -dy) | city.Road(1) | (dx, dy)
```

```
            city.grid[x2, y2] | road | city.grid[x1, y1]
            city.roads[(x2, y2), (x1, y1)] = road

    def __str__(self):
        """Return ascii-art diagram of the planned city."""
        b = textgrid.Box()
        for x, y in self.grid:
            n = len(self.grid[x, y])
            b[(-y, 0), (x, 0)] = {0: ' ', 1: '@', 2: '+', 3: 'O', 4: 'O'}[n]
        for p1, p2 in self.roads:
            d = sub2(p2, p1)
            x, y = p1
            if d == R:
                b[(-y, 0), (x, 2)] = '-'
            else:
                b[(-y, -2), (x, 0)] = '|'
        return str(b)

    def from_str(self, s):
        """Initialize from ascii-art diagram."""
        self.full_plan()
        lines = s.splitlines()
        lines.reverse()
        for y in self.yrange:
            line = lines[y*2]
            for x1, x2 in self.xpairs:
                if len(line) > x1+x2 and line[x1+x2] == ' ':
                    self.del_road((x1, y), (x2, y))
        for y1, y2 in self.ypairs:
            line = lines[y1+y2]
            for x in self.xrange:
                if len(line) > x*2  and line[x*2] == ' ':
                    self.del_road((x, y1), (x, y2))


def turn_probs(ccw_probs, ports=DIRECTIONS):
    """Return turn probs mapping as RandomRouter wants them.

    `ccw_probs` maps from integers (0 = straight, 1 = left,
    2 = -2 = turnaround, 3 = -1 = right) to rotation probabilities.
    `ports` is the list of possible directions.

    """
    probs = {}
    for source in ports:
        probs[source] = {}
    for rotation, prob in ccw_probs.items():
        rotation = rotation % 4          # Normalize direction integers into 0..3 range.
        for source in ports:
            target = source
            for i in range(rotation):
                target = {R: U, U: L, L: D, D: R}[target]
            if target in ports:
                probs[source][target] = prob
    return probs

def uniload(probs):
    """Return turning probs normalized for uniform load.

    For each target direction it normalizes the probabilities of turning to it
    from all source directions to sum up to 1.  Thus, if the rates of cars
    incoming from all directions are equal, the outcoming rates should be
    equal too.

    """
    import copy
    probs = copy.deepcopy(probs)
    sums = {}
    for source in probs:
        for target, prob in probs[source].items():
```

```
            sums[target] = sums.get(target, 0.0) + prob
    for source in probs:
        for target in probs[source]:
            probs[source][target] /= sums[target]
    return probs

def city_queues(city):
    """Return mapping from ((x, y), port) to queue (from city.servers)."""
    res = {}
    for coord, server in city.servers.items():
        for port, queue in server.inputs.items():
            res[coord, port] = queue
    return res


# Test cases:

#c = City()
#c.make_queue = c.LoadStatQueue
#g = GridPlan()
#g.full_plan((10, 10))
#g.sieve_roads(0.3)
#turns = {0: 60, 1: 15, 2: 05, -1: 20}
#g.implement(c,
#            source=lambda : c.ProbSource(0.2),
#            drain=c.Drain,
#            server=lambda ports: c.SwitchedServer(multi.longest_queue_filter),
#            router=lambda ports: c.RandomRouter(uniload(turn_probs(turns, ports))))
#print g
```